

Analysis of Historical Photographs in The Daily Mirror Newspaper

Nesrine Naaman

Sorbonne University

Practice of Image Processing - PRAT

Supervisor:

Isabelle Bloch

February 2026

Table of Content

1	Introduction	2
2	Bibliography	2
2.1	Pipeline	2
2.2	Object Detection Models	2
2.2.1	Classical Detectors : Fast R-CNN	2
2.2.2	RealTime Detectors : YOLO	3
3	Methodology	5
3.1	YOLOv8	5
3.1.1	Architecture	5
3.2	Dataset and Training	6
4	Experimental Results	6
4.1	Model Performance and Metrics	6
4.2	Visual Output Samples	7
5	Discussion and Error Analysis	7
5.1	Detection Challenges	7
5.1.1	Rotation Issue	7
5.1.2	Caption Bounding Box Overlap Issue	7
5.2	Mitigation and Refinement Strategies	9
5.2.1	Handling Rotation	9
5.2.2	Handling Caption BB Overlap	9
6	Limitations and Future Work	11
7	Conclusion	11

1 Introduction

While newspapers are often treated as text-heavy archives, their historical photographs offer a unique visual record that captures nuances text alone frequently misses. This report details the extraction of images and captions of **The Daily Mirror** Newspaper, transforming static scans into a structured, queryable dataset to preserve the vital connection between visual evidence and its original context.

The purpose is to develop an **automated pipeline** for detecting and annotating images and their corresponding captions from The Daily Mirror newspaper pages, enabling the structured digitization and semantic understanding of complex archival layouts.

This task is inherently complex due to the high variability in newspaper layouts, differences in column structure, irregular figure and caption placements, overlapping text, and inconsistencies in historical print quality. As a result, determining not only where each element lies but also which textual region semantically describes a given image becomes a challenging multi-modal problem that combines visual detection, text extraction, and cross-modal understanding.

We will first start by explaining the pipeline that we followed, which was used by most previous work around that area, and then detail some methods and detection models that were present in the literature of historical photograph detection. We will focus on YOLOv8 model and then detail the training process and explain the results we obtained, the flaws, and the approaches taken to solve some of them.

2 Bibliography

2.1 Pipeline

The overall pipeline used in many approaches was divided into several interdependent stages.

First, Object Detection is used to localize visual and textual components, such as images, captions, titles, headlines, and article text, by treating them as objects within the page. In this stage, deep learning-based object detection models such as Faster R-CNN [9] and YOLO [8] play a crucial role in identifying and classifying distinct layout elements.

Second, Optical Character Recognition (OCR) systems like Tesseract OCR [10] are applied to the detected text regions to extract machine-readable textual content.

Third, Both the visual features and textual embeddings are projected into a **shared multi-modal embedding space**. This step enables the system to measure semantic similarity between an image and its caption, and to verify whether a caption correctly describes its associated image.

Finally, Multi-modal reasoning models, such as LayoutLM [13], Donut [3], and OpenCLIP [1], are used to align visual and textual modalities and to infer the most plausible image-caption associations based on both layout proximity and semantic coherence.

This pipeline is the standard pipeline used in The Newspaper Navigator Dataset, and is illustrated in Figure 1 [4].

2.2 Object Detection Models

When dealing with documents and their layout analysis, and in order to deduce the semantics of a particular layout, we can approach the problem by detecting and localizing visual and textual regions as **objects**. This simplifies the task at hand to an object-detection task, which means that we can use pre-existing models that were tailored exactly for that purpose.

2.2.1 Classical Detectors : Fast R-CNN

Previous Deep Learning-based approaches started with **Convolutional Neural Networks (CNNs)**. Thus, we first focus on **CNN-based object detection models** and how they were used for document layout and page region detection.

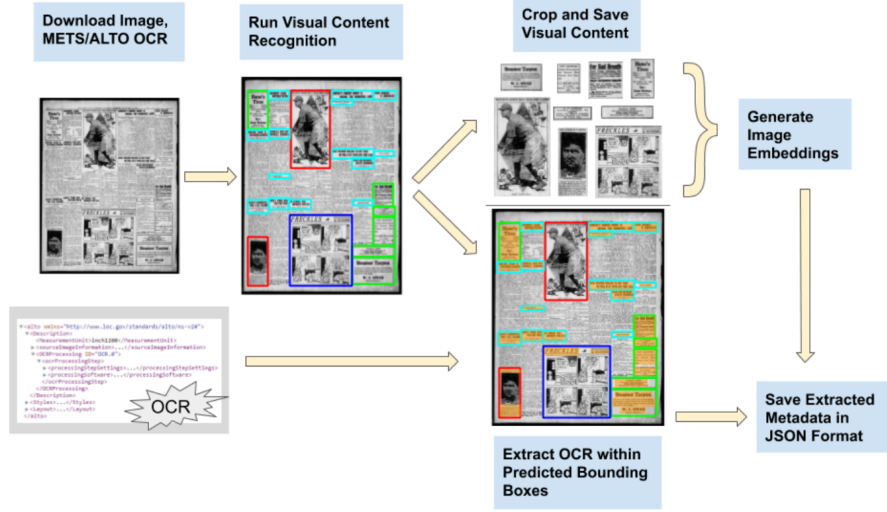
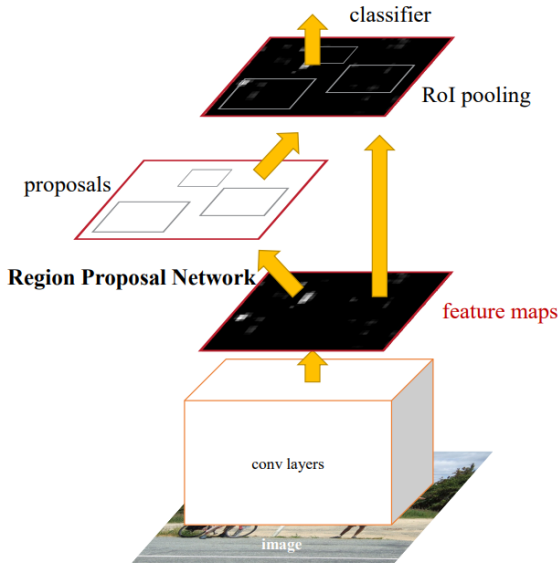


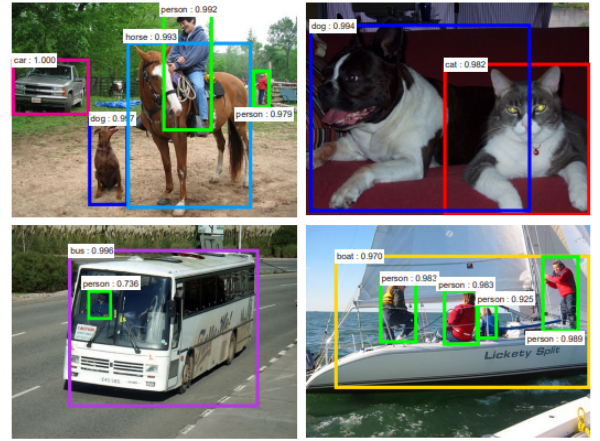
Figure 1: Diagram showing the steps of the pipeline used in the Newspaper Navigator [4]

CNNs were the main tool used to segment the page of a document into meaningful regions. They treated layout detection as a visual object detection problem. Basically, the image is input to the CNN which extracts visual features. Then, segmentation heads identify **bounding-boxes** for each region class.

Faster R-CNN: The idea is that a **Region Proposal Network** (RPN) which is a fully convolutional network, first predicts potential object bounds, each having an objectness score, and then the proposals predicted by the RPN are used by Fast R-CNN for classification (this object is a dog, that object is a person, etc, with a score of confidence) [9]. This is illustrated in Figure 2[9]. This approach was adapted for documents where the objects are often rectangular-shaped layout components instead of natural objects.



(a) Architecture of RPN+Faster R-CNN



(b) Example detections using RPN proposals on PASCAL VOC 2007 test

Figure 2: Architecture and output of Fast R-CNN

2.2.2 RealTime Detectors : YOLO

Real-Time object detectors include the **YOLO (You Only Look Once) model**. This model is basically designed to predict bounding boxes around objects and the objectness score in **one forward pass only** (a single neural network), contrary to previous models that required multiple passes to process and produce the bounding boxes of an image [8]. The main aim of YOLO is the speed in object detection (real-time detection)

while maintaining a good accuracy, and it does so because it treats detection as a **regression problem** rather than a classification one [8].

The YOLO model uses 24 convolutional layers followed by 2 Fully-Connected layers, as seen in Figure 3.

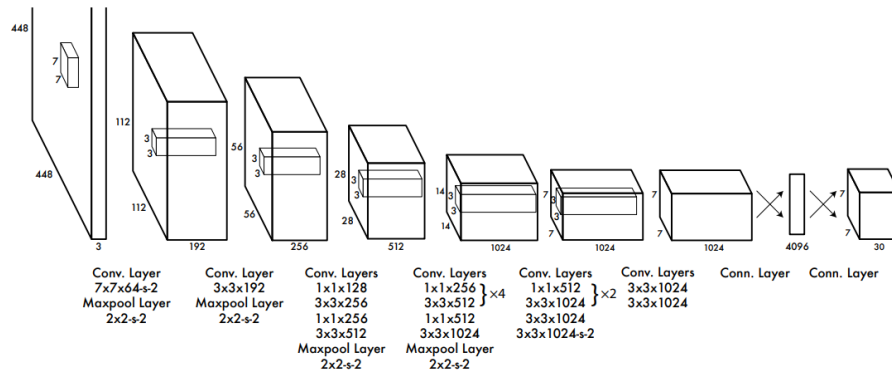


Figure 3: YOLO Architecture [8]

It does the following[8]:

- Splits the image into an $S \times S$ grid
- Each grid cell is responsible for detecting objects whose center falls within the cell
- Each cell: B bounding boxes predicted with a confidence score for each, and C class probabilities per cell (regardless of number of boxes)
- Loss functions: Localization loss (how far predicted box is from ground truth), Confidence loss (how well the model predicts object presence), and Classification loss (how accurate the predicted class is)
- Non-Maximum Suppression: if there are multiple overlapping boxes for the same object, the model keeps only the box with the highest confidence and discards others with a high Intersection-over-Union (IoU) overlap

An example is provided in Figure 4.

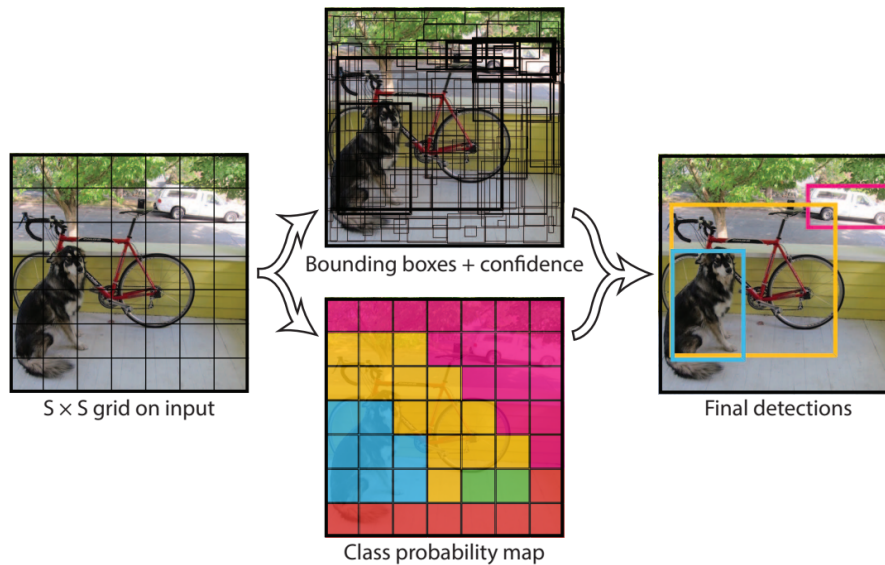


Figure 4: YOLO Application example [8]

3 Methodology

3.1 YOLOv8

Early versions of the YOLO model established the foundation: YOLOv1 introduced the unified architecture [8], while YOLOv2 improved accuracy using anchor boxes and batch normalization [6].

Anchor boxes rely on the idea that before the model even starts detection, there is a pre-defined bounding box with a set width and height at every grid cell of the image. The model then predicts offsets and how the bounding box should be moved in order to fit what was detected in that grid cell, if any.

After that, YOLOv3 added multi-scale predictions to better detect small objects, a feature critical for the dense layouts of newspapers [7]. Subsequent community-led versions like YOLOv5 and YOLOv7 focused on optimizing the speed-to-accuracy trade-off through more architectural refinements [11].

For the analysis of the Daily Mirror, YOLOv8 was a superior choice because it transitions to an **anchor-free** architecture [14]. In historical newspapers, captions and photographs vary wildly in aspect ratio and size; traditional anchor-based models struggle with these non-standard shapes, whereas YOLOv8's anchor-free approach directly predicts the center of the object, making it significantly more flexible for complex, unstructured layouts. Furthermore, v8 integrates with Vision Transformers for enhanced context awareness, which helps in layout understanding tasks [12].

3.1.1 Architecture

In the architectural design of YOLOv8, the model is divided into three distinct components: the backbone, the neck, and the head, which together facilitate high-performance real-time detection [11]. Figure 5 shows the architecture of YOLOv8 including the aforementioned 3 components [11].

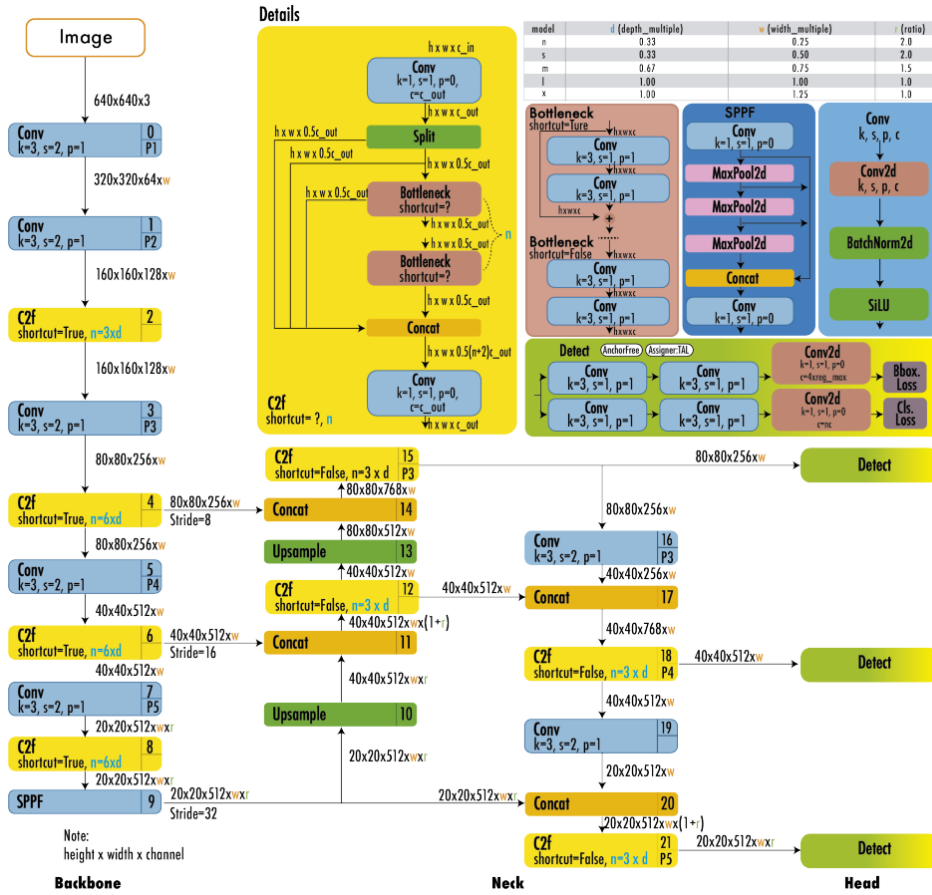


Figure 5: YOLOv8 Architecture [11]

The backbone serves as the primary **feature extraction network**, utilizing a deep convolutional structure to extract hierarchical features ranging from low-level edges and textures to high-level semantic object repre-

sentations. These features are then passed to **the neck**, which employs a Path Aggregation Network (PANet) and Feature Pyramid structures to gather information across different scales, ensuring that both small captions and large photographs are accurately represented [11].

The head of the model implements an anchor-free detection strategy using a decoupled architecture, where classification and regression tasks are handled by separate branches. For every cell in the output grid, the head predicts the bounding box coordinates, class probabilities, and objectness confidence scores. To optimize these predictions, YOLOv8 utilizes a sophisticated loss function: Binary Cross-Entropy (BCE) is used for classification, while Complete Intersection over Union (CIoU) and Distribution Focal Loss (DFL) are employed for bounding box regression [11].

3.2 Dataset and Training

Our dataset consisted of 1000 Daily Mirror newspaper pages. 197 pages were annotated, and the rest non-annotated.

The annotation was in the form of a **JSON** file that followed the **COCO** format [5]. This means that we had a dictionary of information about the newspaper pages (their id, file name, width, height, etc.), as well as information about the annotations of each page (annotation id, image id that it belongs to, category id to specify the class of the annotation, bounding box coordinates, and more). The classes of annotations included: Sub-caption, Caption, Title, Image, and Sub-image.

YOLOv8 however, accepted a different format of annotations. It only accepted **yolo_labels**, which are txt files representing the annotations of each newspaper page, with each annotation being a line in the file containing information about the category of the object as well as its bounding box coordinates only. For that reason, a **pre-processing** step was necessary to modify the format of the annotations to one that was YOLOv8-ready.

Furthermore, since we did not have much annotated data, it was necessary that we use a YOLOv8 model that was already **pre-trained** on COCO data beforehand [5], to be able to benefit from **Transfer learning** and what the model has already learned about features of images [2].

We split our annotated data into 80% training and 20% validation, and we fully fine tuned the pre-trained YOLOv8 model. The results we obtained from the model were then transformed into a JSON file of annotations similar to the one we had for the 197 annotated data we started with.

4 Experimental Results

4.1 Model Performance and Metrics

The Ultralytics library already provides measures of the model’s Mean Precision, Mean Recall, Mean Average Precision (mAP) calculated at a fixed IoU threshold of 0.50 (mAP@50), and mAP calculated at different IoU thresholds ranging from 0.50 to 0.95 (mAP@50-95). The metrics can be seen in the following table:

Table 1: YOLOv8 Validation Metrics for Daily Mirror Analysis

Class	Instances	Precision	Recall	mAP50	mAP50-95
All	504	0.944	0.735	0.732	0.531
Image	236	0.979	0.989	0.992	0.932
Caption	233	0.940	0.953	0.976	0.611
Title	32	0.858	1.000	0.956	0.579
Subimage	3	1.000	0.000	0.004	0.003

The library also outputs confusion matrices. The normalized confusion matrix we obtained from our model is the following:

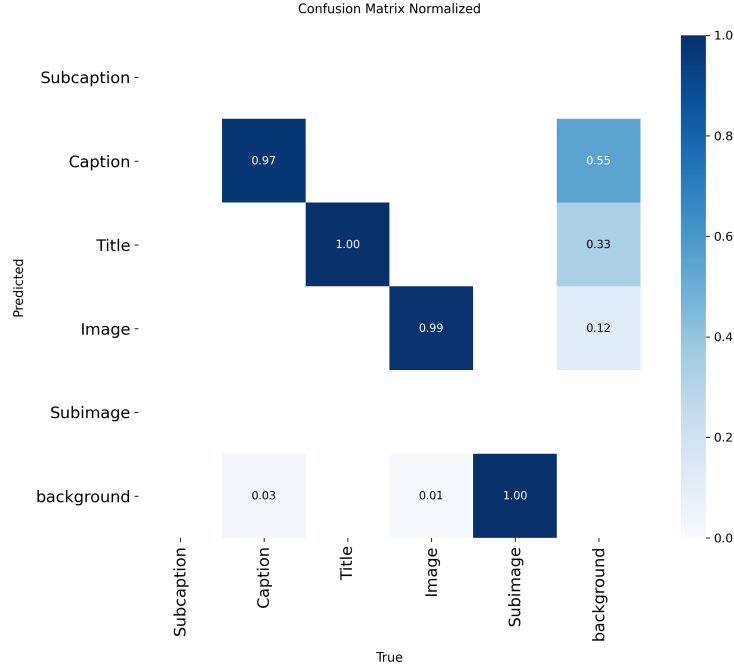


Figure 6: Normalized Confusion Matrix of the YOLOv8 Model

4.2 Visual Output Samples

We present in Figure 7 some of the detections of non-annotated data given by our model at inference time. We can clearly see that the model has successfully identified photos and captions in standard layouts with high confidence scores and good bounding box placements. Some titles were also successfully detected, although their confidence scores tended to decrease as their placement moved lower on the page.

5 Discussion and Error Analysis

From Figure 6 and table 1, we can see that the model gave a good score for detecting Images, with a mAP50-95 of 0.932, which is a great result. This suggests that the visual contrast between a photograph and the newspaper background is so high that YOLOv8 can define the boundaries with near pixel-perfect accuracy.

For Titles, the model found every single title in our test set (Recall=1). For Captions, the mAP50 is high (0.976), but the mAP50-95 drops to 0.611. This indicates that while the model always finds the caption, it struggles slightly to define the exact tight boundary.

Although the results we obtained were very accurate and the model performed really well, we still faced some challenges: The model was not able to detect rotated images, and for some pages, it was outputting many overlapping caption bounding boxes for just one caption (possibly explaining the lower mAP50-95 percentage that we obtained).

5.1 Detection Challenges

5.1.1 Rotation Issue

For the rotation issue, it was clear to us why the model failed at detecting slightly rotated photographs. Since YOLOv8 typically uses axis-aligned bounding boxes and since there were no rotated photographs present in the training newspaper pages, it is evident that it will not be able to detect the couple of rotated scans that were in the non-annotated Daily Mirror archive. An example of an un-detected rotated photograph can be seen in Figure 8.

5.1.2 Caption Bounding Box Overlap Issue

For the caption issue, we faced some cases where the model gave many bounding boxes that were overlapped around one caption that was usually double-columned. Examples of those cases can be seen in Figure 9.

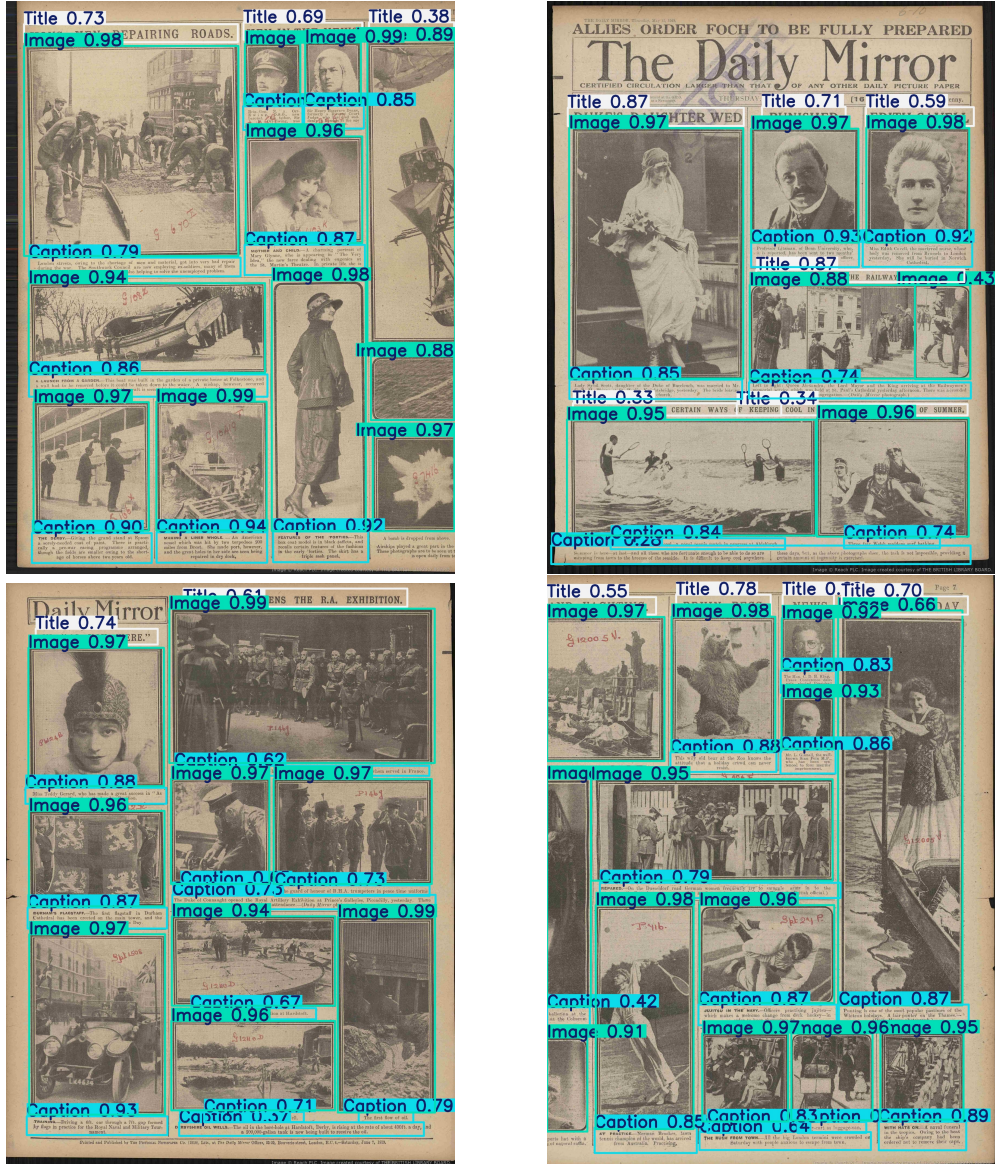


Figure 7: Detection Examples of Non-Annotated Data

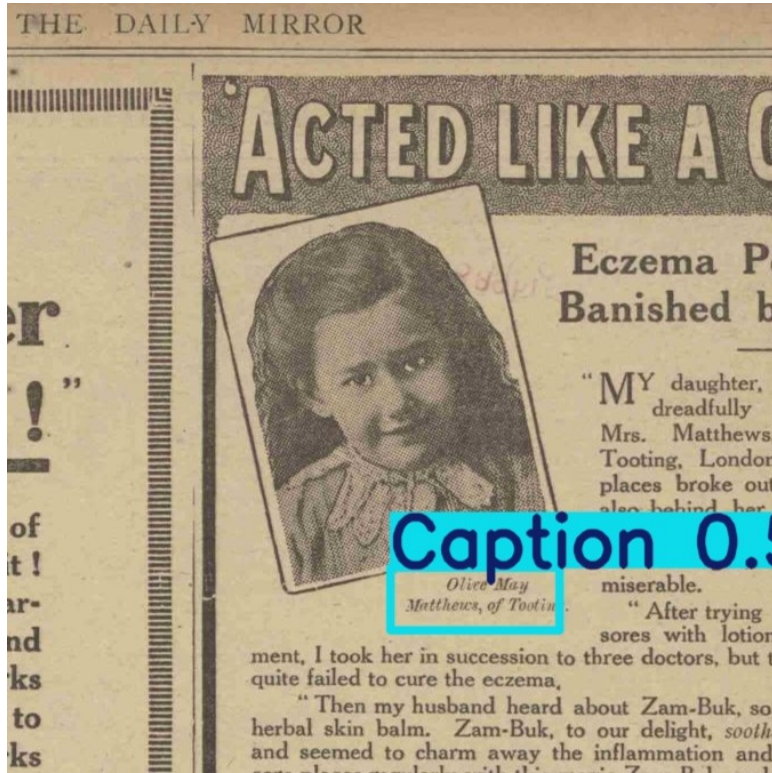


Figure 8: Rotated Photograph not detected by YOLOv8

5.2 Mitigation and Refinement Strategies

5.2.1 Handling Rotation

To solve the rotation issue, we thought a good approach would be to train the model on rotated photographs to be able to detect them at inference time. Therefore, we carried out **Data Augmentation**.

First, We synthesized new newspaper images by **rotating** the last photograph in every page we had in our training and test sets. The angle according to which the photographs were rotated was chosen randomly among the 6 following angles: $[-15, -10, -5, +5, +10, +15]$, since we assumed that a photograph in the newspaper would not be rotated more than that.

After rotating the patch, the pixels right outside the patch on the sides were assigned a "background color", chosen to be the color right outside the photograph's bounding box before rotation.

Furthermore, we replicated the same annotations of that newspaper page, while **only** modifying the bounding box coordinates of the rotated photograph, to be now a bigger box encapsulating all the 4 corners of the rotated patch. A visual example of the new bounding box detecting a rotated patch is provided in Figure 10.

Finally, we got the total augmented dataset -which contained all the previous training and test set plus the new synthesized pages with rotated photographs- and we fully fine-tuned another pre-trained YOLOv8 model, to result with an **improved YOLOv8 model** that could now detect rotated images.

Indeed, when we put that new model to test, it was successfully able to detect rotated images in the test set, as seen in Figure 11, but most importantly, it was able to also detect the problematic rotated photograph that we started with in Figure 8. The detection can be seen in Figure 12 showing that Data Augmentation has helped us solve this issue successfully.

5.2.2 Handling Caption BB Overlap

Regarding the overlap of some caption bounding boxes, we decided to carry out a post-processing step where we just looped through the annotated results that the model gave and if 2 captions bounding boxes had an overlap of more than 10% (IoU), we just merged their bounding boxes into one, and updated the annotation accordingly. The results of the merging can be seen in Figure 13.

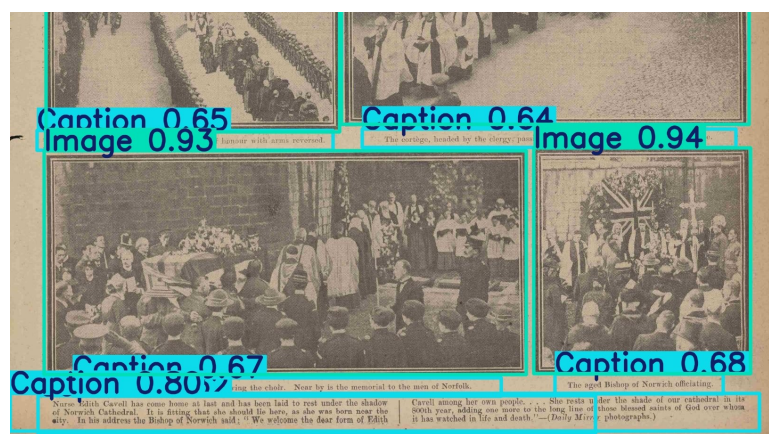
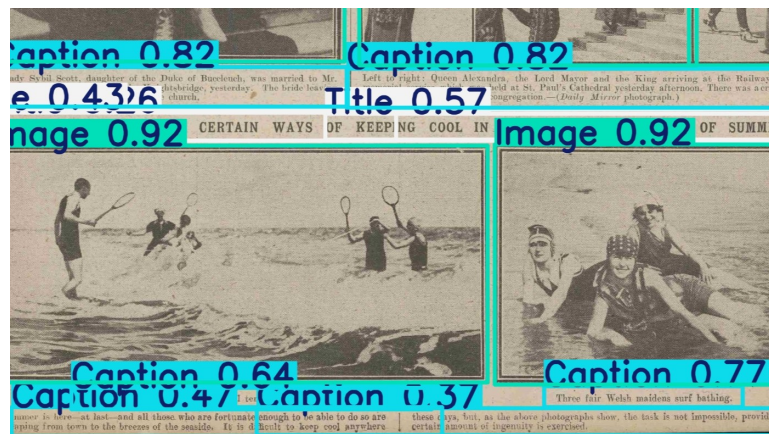
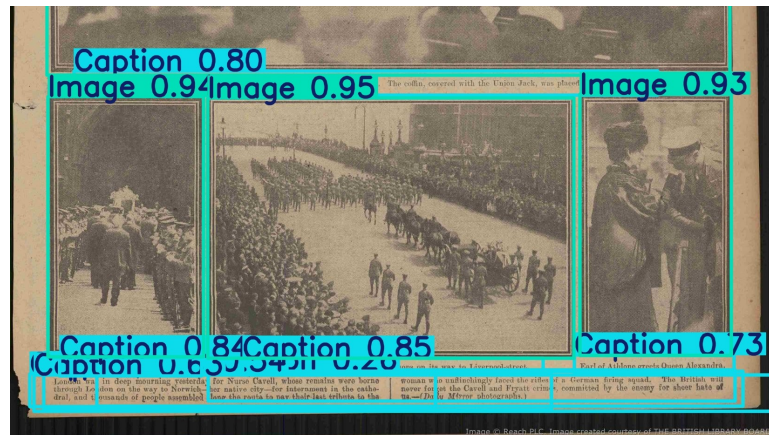


Figure 9: Overlapping Caption Bounding Boxes



Figure 10: New Bounding Box detecting a rotated photograph

6 Limitations and Future Work

While the current work successfully identifies key layout elements, a notable limitation would be in the handling of complex, multi-column captions. Currently, we merged overlapping bounding boxes into a single larger region. However, if the captions were split into two columns by a vertical rule, a more robust approach for downstream Optical Character Recognition (OCR) would be to detect and split these into two distinct bounding boxes rather than a single merged entity. This would preserve the natural reading order and prevent the OCR engine from interleaving text from different columns, which is a common source of semantic noise in digitizing historical archives.

Looking forward, the next phase of this research involves a transition from structural detection to semantic understanding. The primary objective as we mentioned in the pipeline in section 2.1 [4] is to implement OCR to extract raw text, followed by the projection of both visual features and textual embeddings into a shared multi-modal embedding space. This step will enable the system to measure the semantic similarity between an image and its caption, providing a quantitative method to verify whether a caption accurately describes its associated photograph.

Finally, the last step would be to integrate multi-modal reasoning models such as LayoutLM [13], Donut [3], and OpenCLIP [1]. These architectures will be used to align visual and textual modalities, allowing the system to infer the most plausible image-caption associations by balancing spatial layout proximity with deep semantic coherence.

7 Conclusion

In conclusion, this report emphasizes the critical importance of knowing our data thoroughly and understanding the diverse layouts encountered in historical newspapers to tailor a model’s behavior specifically to the task at hand. While transfer learning and fine-tuning are powerful, they are not enough on their own; we must constantly identify unique issues and approaches that emerge from our specific domain. Given that our task involves historical photographs, it was essential to match the model to the complex design variations of Daily Mirror layouts (such as rotated images and double columned captions).

Moreover, we saw how object detection serves as an extremely important foundation, laying the groundwork for subsequent OCR and multi-modal embedding. This pipeline is necessary to transform a photograph from a mere static scan into a “living query”, where semantic meaning is extracted from both the image and its associated caption. This domain is invaluable for historians, as it allows them to query digital archives for specific information about historical events with unprecedented ease. By making these archives searchable and structured, we enable historians to extract brand-new information about significant events and answer questions that were previously deemed unanswerable.

Code and Data Availability

The implementation of the YOLOv8 pipeline, along with the configuration files and pre-processing/post-processing scripts, is available at the following GoogleDrive link: <https://bit.ly/4kHL33F>.

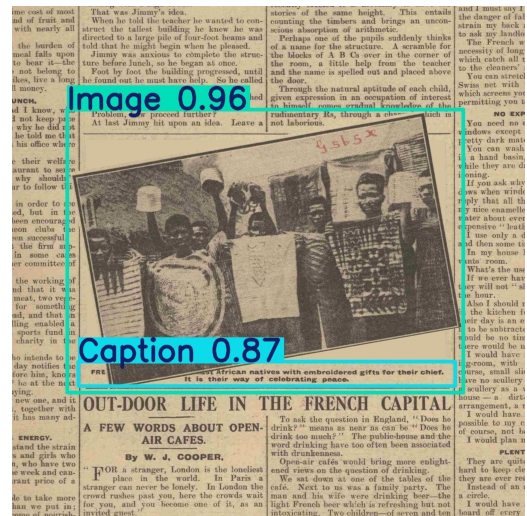
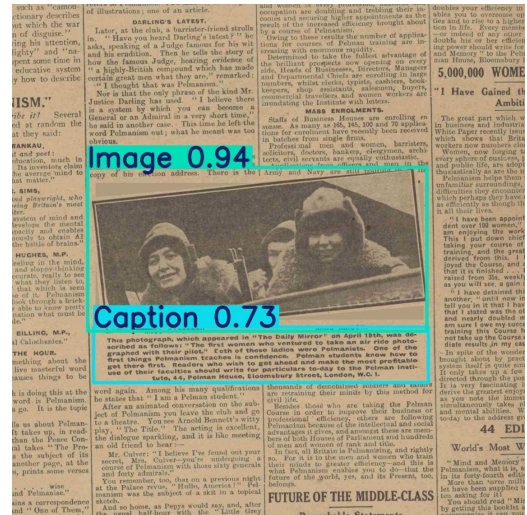
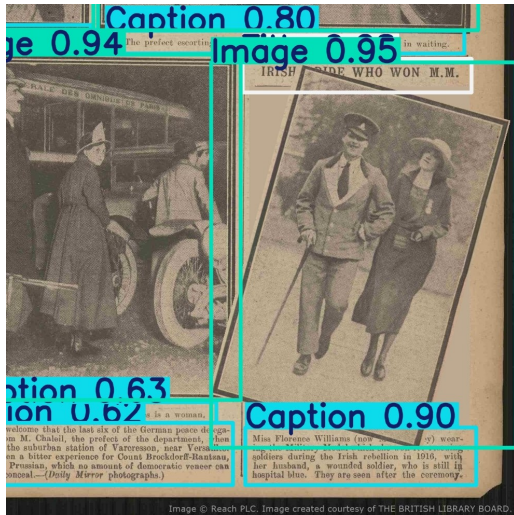


Figure 11: Improved YOLOv8 Detecting Rotated Photographs in the Augmented Test Set

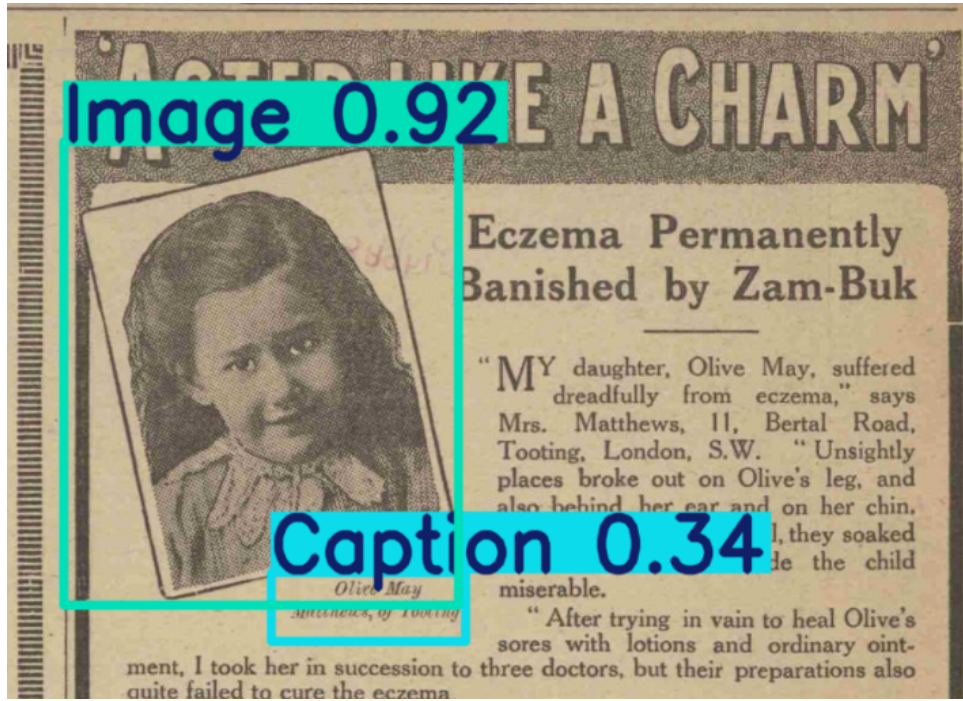
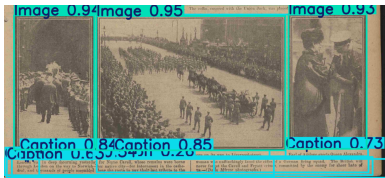
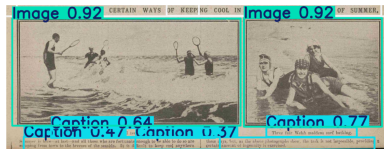


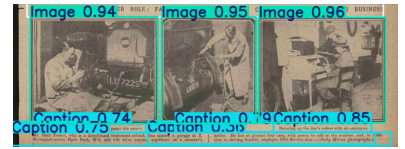
Figure 12: The un-annotated Rotated Photograph successfully detected by YOLOv8 after Data Augmentation



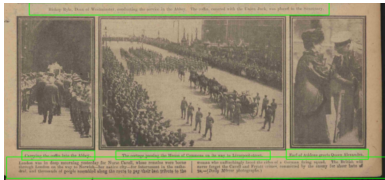
(a) First Example Before Merging



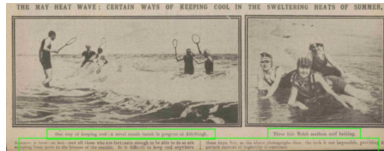
(b) Second Example Before Merging



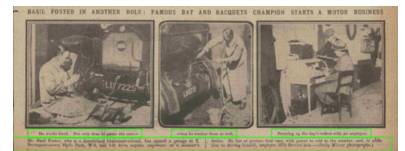
(c) Third Example Before Merging



(d) First Example After Merging



(e) Second Example After Merging



(f) Third Example After Merging

Figure 13: Some Examples before and after merging their overlapping caption bounding boxes

Acknowledgements

We would like to express our sincere gratitude to the historian team at Université Paris Cité **Dr. Mohamed Chelali** and **Dr. Daniel Foliard** for providing the Daily Mirror dataset and the expert annotations that served as the foundation for this research.

References

- [1] Gabriel Ilharco et al. *OpenCLIP*. Version 0.1. If you use this software, please cite it as below. July 2021. DOI: 10.5281/zenodo.5143773. URL: <https://doi.org/10.5281/zenodo.5143773>.
- [2] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8*. Version 8.0.0. 2023. URL: <https://github.com/ultralytics/ultralytics>.
- [3] Geewook Kim et al. “OCR-Free Document Understanding Transformer”. In: *Computer Vision – ECCV 2022*. Lecture Notes in Computer Science. Springer, 2022, pp. 498–517. DOI: 10.1007/978-3-031-19815-1_29.
- [4] Benjamin C. G. Lee et al. “The Newspaper Navigator Dataset: Extracting Headlines and Visual Content from 16 Million Historic Newspaper Pages in Chronicling America”. In: *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. New York, NY, USA: ACM, 2020, pp. 3055–3062. DOI: 10.1145/3340531.3412767.
- [5] Tsung-Yi Lin et al. “Microsoft COCO: Common Objects in Context”. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2014, pp. 740–755. DOI: 10.1007/978-3-319-10602-1_48.
- [6] Joseph Redmon and Ali Farhadi. “YOLO9000: Better, Faster, Stronger”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 7263–7271. DOI: 10.1109/CVPR.2017.690.
- [7] Joseph Redmon and Ali Farhadi. “YOLOv3: An Incremental Improvement”. In: *arXiv preprint arXiv:1804.02767* (2018). URL: <https://arxiv.org/abs/1804.02767>.
- [8] Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 779–788. DOI: 10.1109/CVPR.2016.91.
- [9] Shaoqing Ren et al. “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks”. In: *Advances in Neural Information Processing Systems 28*. 2015, pp. 91–99.
- [10] Ray Smith. “An Overview of the Tesseract OCR Engine”. In: *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*. Curitiba, Brazil, 2007. DOI: 10.1109/ICDAR.2007.4376991.
- [11] Juan Terven and Diana Cordova-Esparza. “A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS”. In: *Machine Learning and Knowledge Extraction* 5.4 (2023), pp. 1680–1716.
- [12] Rejin Varghese and Sambath M. “YOLOv8: A Novel Object Detection Algorithm with Enhanced Performance and Robustness”. In: *2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)*. 2024, pp. 1–6. DOI: 10.1109/ADICS58448.2024.10533619.
- [13] Yiheng Xu et al. “LayoutLM: Pre-training of Text and Layout for Document Image Understanding”. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD ’20. Virtual Event, CA, USA: Association for Computing Machinery, 2020, pp. 1192–1200. ISBN: 9781450379984. DOI: 10.1145/3394486.3403172. URL: <https://doi.org/10.1145/3394486.3403172>.
- [14] Muhammad Yaseen. “What is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector”. In: *arXiv preprint arXiv:2408.15857* (2024). URL: <https://arxiv.org/abs/2408.15857>.